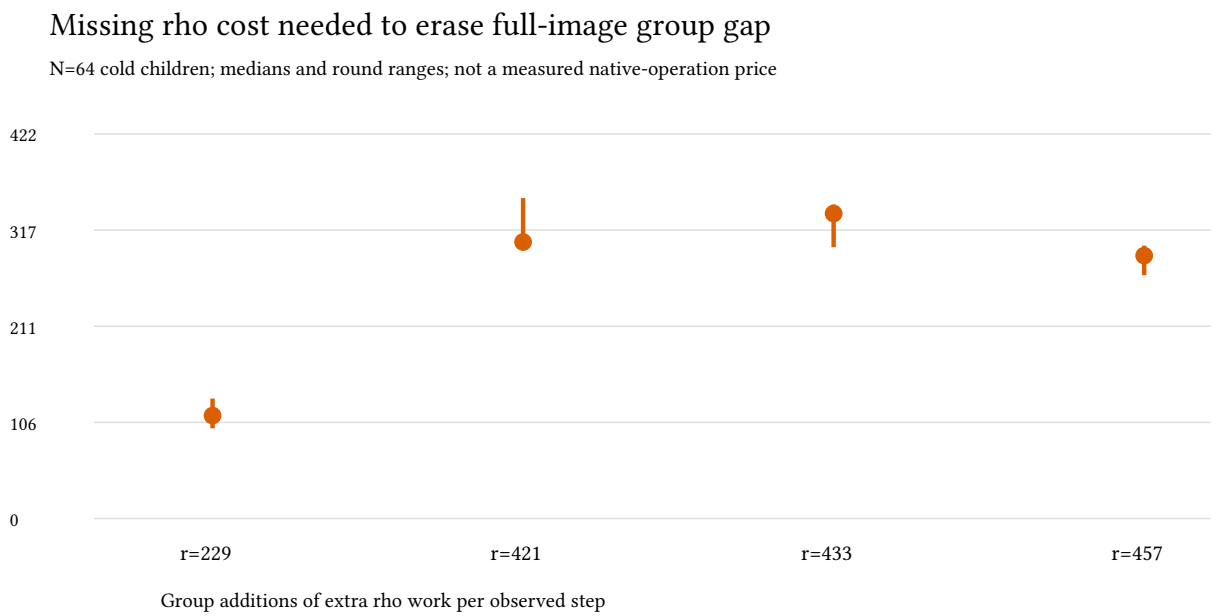
Native price-sensitivity derivation: run record

Date and host: 2026-10-15, Darwin arm64, shared L0. This is a deterministic analysis of previously frozen counts. Its elapsed time is not a performance measurement.

The protocol `PROTOCOL_V1.md` froze four run SHA-256 values, their independent replay certificates, 48 paired cells and the formula before the calculation. The Rust source is `examples/different_j_price_sensitivity.rs`. The build and run command appear verbatim in `RUN.md`.

The executable exited 0. `SENSITIVITY_V1.exit`, `.stdout` and `.stderr` retain the process receipts. The native analyzer checked the four complete run hashes, matching replay digests and pass receipts, 84 child keys and 1,785 correct scalar outputs per order, all 48 corresponding group totals from `../TREND_V1.json`, and positive rho step totals. `SENSITIVITY_V1.json` contains all 48 paired rows and the four N64 summaries; `RESULTS.pdf` interprets them.



The graph shows the additional omitted rho work per observed step needed to reach the full-image group-count lower bound when every omitted full-image cost is set to zero. It is a sensitivity threshold, not a measured operation price. No child solver, target, cap or frozen input changed. Disjoint native-operation accounting, cross-host calibration, independent operation traces, isolated timing and second-host performance remain separate gates.